

Part 2: Infrastructure Design

1. Compute Choice: Cloud Run

For a startup, Cloud Run is the absolute best choice. It is a fully managed serverless platform for containerized applications.

- Why not GKE (Kubernetes)? GKE is incredibly powerful but comes with a steep learning curve, operational overhead, and a baseline cluster management fee. It is overkill for early-stage startups.
- Why not Compute Engine (VMs)? Managing VMs means patching OS vulnerabilities, configuring Managed Instance Groups (MIGs) for scaling, and paying for idle compute when traffic is low.
- Why Cloud Run? * Cost: strictly pay for the exact compute time used (down to the 100-millisecond). It can scale to zero when there is no traffic, costing nothing.
 - Auto-scaling: It scales up automatically and instantly based on incoming concurrent requests.
 - Simplicity: If an app can run in a Docker container, it can run on Cloud Run.

2. MongoDB Hosting Approach: MongoDB Atlas (on GCP)

Do not host and manage own database on Compute Engine VMs or Kubernetes. The operational burden of managing backups, replicas, and failovers is a massive drain on startup engineering resources.

- The Solution: Use MongoDB Atlas, officially deployed on GCP.
- Cost-efficiency: Atlas offers a Serverless tier or small dedicated clusters (starting very cheaply) that can scale up with a single click as the user base grows.
- Security: can set up VPC Peering or use Private Endpoint (PrivateLink) between the GCP VPC and the MongoDB Atlas VPC. This ensures database traffic never travels over the public internet.

3. Networking & Secure Access

To keep infrastructure secure while serving public traffic, we need to separate ingress (incoming traffic) from egress (outgoing traffic to the database).

- Ingress (Incoming): Put a Global HTTP(S) Load Balancer in front of Cloud Run. This gives :
 - Free, managed SSL certificates.
 - A single static IP for a custom domain.
 - Integration with Cloud Armor (Google's Web Application Firewall) to block DDoS attacks, SQL injection, and rate-limit abusive IPs.

- Egress (Outgoing): Use Direct VPC Egress for Cloud Run service. This attaches a serverless app directly to a custom VPC, allowing it to communicate with the MongoDB Atlas database securely via internal private IPs.

4. Secrets & IAM

Security in GCP relies heavily on the principle of least privilege. Do not hardcode database URIs or API keys in code or environment variables.

- Secrets: Store MongoDB connection string and third-party API keys in Google Secret Manager. Cloud Run integrates natively with it, mounting secrets as environment variables securely at runtime.
- IAM (Identity and Access Management): Create a dedicated Service Account specifically for this Cloud Run service. Grant this service account *only* the permissions it needs (e.g., [Secret Manager Secret Accessor](#)). This ensures that even if the application is compromised, the blast radius is strictly contained.

5. Logging & Monitoring Stack

Skip the complex setups like ELK (Elasticsearch, Logstash, Kibana) or Prometheus/Grafana for now.

- The Solution: Use Google Cloud Observability (formerly Stackdriver).
- Because we are using Cloud Run, logging and monitoring are completely zero-config. Standard output (`stdout/stderr`) from containers is automatically parsed and searchable in Cloud Logging.
- Cloud Monitoring provides out-of-the-box dashboards for CPU utilization, memory usage, request latency, and 5xx error rates. can easily set up alerts to ping Slack/email if errors spike or if monthly billing exceeds a certain budget.

- Architecture diagram

GCP Architecture for Startup App (Cloud Run + MongoDB Atlas)

Optimized for auto-scaling, secure access and reasonable cost. Unceaed, secure access reasonable cost.

